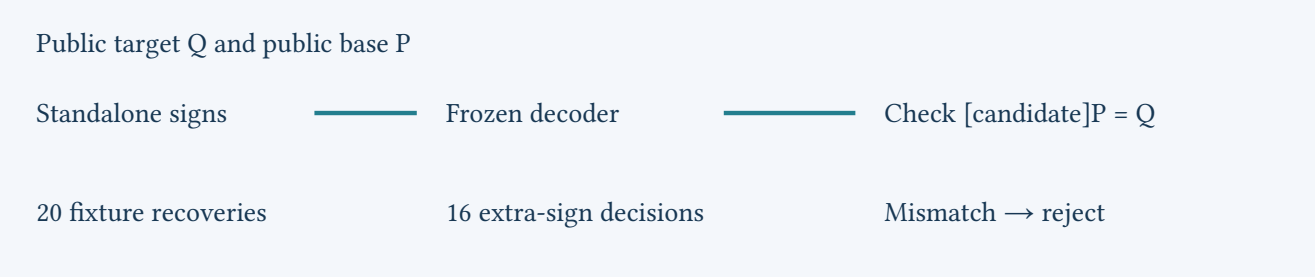
Standalone public-point recovery

T-1789 • 2026-11-07 • PCC-20261107-58f49b (unsigned)

The recovery API now connects projected character queries to the frozen quartic/bitplane decoder and checks the returned scalar against the supplied public point before returning. A candidate mismatch produces a verification error inside the API.



Construction

The compiler prefix is preserved with its source module removed; generation.json binds its input and output. Public shifts produce the initial sign window and any requested additional sign. The initial target is validated before affine addition. Binary double-and-add verifies the candidate. No target scalar label enters the runtime API.

Saved software evidence

Two selected native test methods pass; 117 imported methods are filtered out. The frozen order479 fixture supplies20 public points, including identity and the16 exceptional cases needing an extra sign. All recoveries, extra offsets and sign-query counts match. Changed candidates and a deliberately corrupted decoder reject. Domain, window and coordinate controls pass.

The normal metadata build excludes six legacy source/transport paths. All32 dependency hashes still match after migration. Test and normal builds retain14 and24 unused-code warnings. No build or test failed in validation-01. These are producer checks, with shared arithmetic, not independent verification or a timing panel.

Scope and next step

The source inherits caller-certified prime parameters and projection order2 or4. The original4563-case cohort is unchanged. Each sign still constructs its geometry/cache; prepared-window sharing remains the next integration. The compiler still enumerates the public scalar field, and geometry retains its existing square-root-scale construction.

Evidence and references

README.md specifies the API/domain and replay command; validation-01/receipt.json pins commands and dependencies. generation.json records the extraction. The mathematical basis is the preceding standalone-source derivation and Ward identities. Rust dep-info semantics are documented at https://doc.rust-lang.org/rustc/command-line-arguments.html. No cloud experiment was launched.